

Description “Commit3”

- มีการเขียนการทำงานโค้ดเพิ่มเติม ใน function ต่อไปนี้

Piece.draw() - เพื่อวาดชิ้นส่วนในมือ

spawn_hand() - เพื่อสุ่มสร้างชิ้นส่วน 3 ชิ้นในมือ

จากนั้นกลับไปเขียนเพิ่มเติมใน setup() และ draw()

Piece.draw(self):

หลักการทำงาน

แบ่งออกเป็น 2 ส่วนหลัก

ส่วนที่ 1 คือการเตรียมค่าที่ต้องใช้ก่อนเข้าสู่วาด ได้แก่การหีบสีของชิ้นส่วนจาก PALETTE[self.color_idx] มาเก็บไว้ (block_color) พร้อมกำหนดสีขอบตายตัวเป็นสีดำ (edge_color) และตัดสินใจผ่านเงื่อนไข if self.is_dragging ว่าจะใช้ขนาดพิกเซลต่อช่องเท่าไร (unit) โดยตั้งเงื่อนไขดังนี้ ถ้ากำลังถูกลากอยู่ใช้ขนาดเท่ากระดานจริง (CELL_SIZE) เพื่อให้ผู้เล่นเห็นภาพใกล้เคียงกับที่จะวางจริง แต่ถ้ายังอยู่ในมือเฉยๆ ใช้ขนาดย่อส่วน (self.mini_cell) เพื่อประหยัดพื้นที่แสดงผล

ส่วนที่ 2 คือการวนลูปไล่วาดทีละบล็อกตามรูปทรงที่เก็บใน self.blocks จนครบทุกบล็อก โดยแต่ละรอบจะดึงพิกัดของบล็อกนั้น (offset) มาแปลงเป็นตำแหน่ง pixel จริงบนจอ ซึ่งกำหนดค่าให้ตัวแปร bx , by ด้วยการคูณกับ unit แล้วบวกกับตำแหน่งปัจจุบันของชิ้นส่วนทั้งก้อน (self.x, self.y) ก่อนเรียก draw_square() วาดบล็อกนั้นออกมา ทำซ้ำแบบนี้จนกว่าจะวาดครบทุกบล็อกตามจำนวนที่มีอยู่ในรูปทรงนั้นๆ

spawn_hand():

หลักการทำงาน

แบ่งออกเป็น 3 ส่วนหลัก

ส่วนที่ 1 คือการเตรียมค่าพื้นฐานก่อนเข้าสู่ได้แก่ slot_w ซึ่งคำนวณจากการหารความกว้างหน้าจอออกเป็น 3 ส่วนเท่าๆ กัน (เพราะมีชิ้นส่วนในมือ 3 ชิ้นพอดี) และ slot_x ซึ่งเป็นตัวแปรสะสมตำแหน่งเริ่มต้นของแต่ละสล็อต เริ่มต้นที่ 0 คือขอบซ้ายสุดของจอ

ส่วนที่ 2 คือการวนลูปสร้างชิ้นส่วนใหม่ที่ละชิ้นจนครบทุกช่องใน hand ซึ่งภายในลูปนี้แบ่งการทำงานย่อยออกเป็นลำดับ ดังนี้

ขั้นแรก สุ่มเลือกรูปทรงและสีจาก SHAPE_TEMPLATES ด้วย random.choice() แล้วแยกเก็บไว้ในตัวแปร shape และ hue

ขั้นที่สอง หาความกว้างจริงของชิ้นส่วนที่สุ่มได้ ด้วยการวนลูปซ้อนอีกชั้นหนึ่งไล่ดูทุกบล็อกในรูปทรงนั้น เพื่อหาค่าคอลัมน์ (แกน x) ที่อยู่ขวาสุด (max_col) แล้วนำไปคำนวณความกว้างจริงเป็น pixel (piece_width) เหตุผลที่ต้องหาความกว้างจริงของแต่ละชิ้น เพราะชิ้นส่วนแต่ละรูปทรงมีขนาดไม่เท่ากัน การจัดให้อยู่กึ่งกลางสล็อตพอดีจึงต้องรู้ความกว้างที่แท้จริงก่อน ไม่สามารถใช้ค่าคงที่ค่าเดียวกับทุกรูปทรงได้

ขั้นที่สาม คำนวณตำแหน่ง new_x โดยนำตำแหน่งกึ่งกลางของสล็อต (slot_x + slot_w/2) มาลบด้วยครึ่งหนึ่งของความกว้างจริงที่เพิ่งคำนวณได้ (piece_width/2) เพื่อให้ชิ้นส่วนอยู่กึ่งกลางสล็อตพอดี ส่วน new_y กำหนดเป็นค่าคงที่ตายตัว เพราะชิ้นส่วนในมือทุกชิ้นวางอยู่แถวเดียวกันเสมอ

ขั้นสุดท้าย สร้าง object Piece ใหม่จากค่าทั้งหมดที่เตรียมไว้ (shape, hue, new_x, new_y) แล้วเก็บลงในช่องที่ i ของ hand

ส่วนที่ 3 คือการเลื่อนตำแหน่งเตรียมพร้อมสำหรับสล็อตถัดไป โดยบวกสะสม slot_x += slot_w เพื่อขยับไปยังจุดเริ่มต้นของสล็อตถัดไป และเพิ่มค่า i เพื่อขยับไปสร้างชิ้นส่วนในช่องถัดไปของ hand

Description “Commit 4”

- มีการเขียนการทำงานโค้ดเพิ่มเติม ใน function ต่อไปนี้

Piece.contains_point() - เพื่อเช็คความคลิกโดนแต่ละ block ในชิ้นส่วนใหม่

mousePressed() - วนหาชิ้นส่วนที่เมาส์คลิกโดน (ผ่าน contains_point) และเริ่มสถานะลาก

mouseDragged() - เมื่อมีการลากเมาส์จะอัปเดตตำแหน่ง x, y ของชิ้นที่โดนคลิกให้เคลื่อนตามตำแหน่งเมาส์ปัจจุบัน

Piece.reset_pos() - ชิ้นส่วนกลับคืนตำแหน่งเดิมหากวางไม่ได้

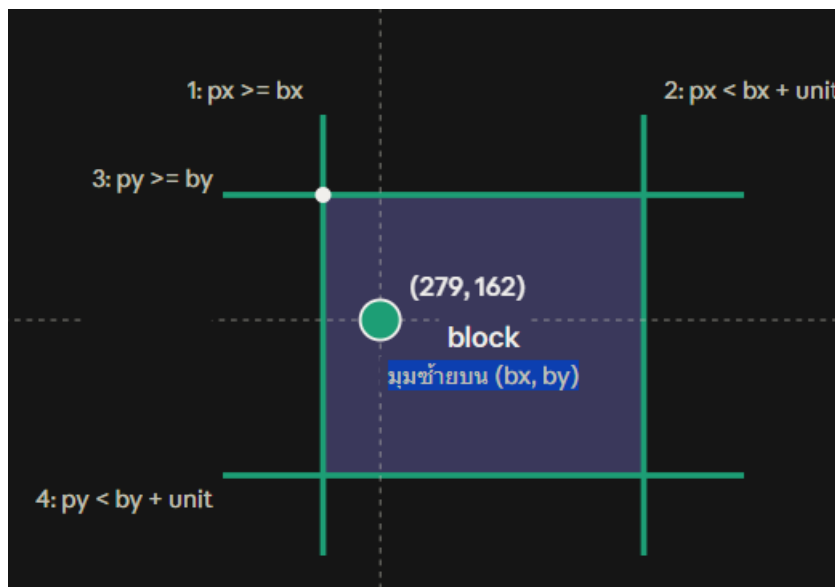
Piece.contains_point(self, px, py):

ตรวจว่าจุด (px, py) ตกอยู่บนชิ้นส่วนหรือไม่ แล้วคืนค่า True หรือ False

1. **เลือกขนาดช่อง** ใช้ CELL_SIZE (48 px) ถ้ากำลังลาก และ mini_cell (24 px) ถ้าอยู่ในมือ เพื่อให้พื้นที่ตรวจตรงกับที่ draw() วาดบนจอ
2. **วนตรวจทีละ block** โดยมีการแปลงพิกัดช่องของ block เป็น pixel self.x + offset[0] * unit และ self.y + offset[1] * unit จะได้พิกัดมุมซ้ายบนของ block นั้น
3. **เช็คจุด px, py** ซึ่งก็คือตำแหน่งเมาส์ตอนคลิก อยู่ในพื้นที่ของ block หรือไม่ โดยต้องเป็นจริงพร้อมกัน 4 เงื่อนไข คือ
 1. $px \geq bx$ ต้องไม่เกินตำแหน่งขอบซ้าย
 2. $px < bx + unit$ ต้องไม่เกินตำแหน่งขอบขวา
 3. $py \geq by$ ต้องไม่อยู่เหนือขอบบน
 4. $py < by + unit$ ต้องไม่อยู่ต่ำกว่าขอบล่าง

ถ้าเป็นจริงทั้งหมดให้ return True ทันที

4. **กรณีไม่พบเลย** เมื่อวนครบทุก block แล้ว ไม่มี block ใดโดนคลิก จึง return False

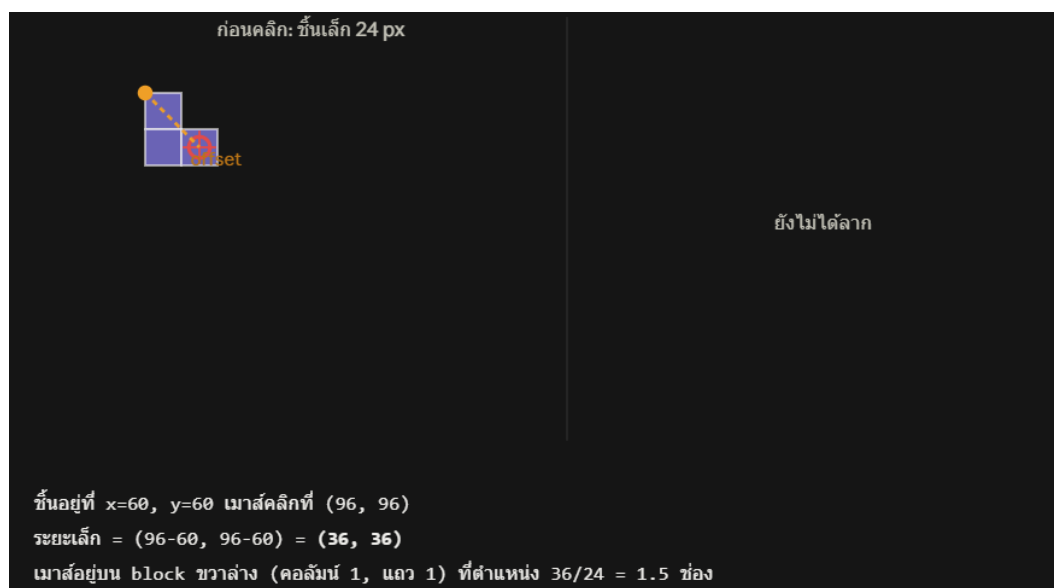


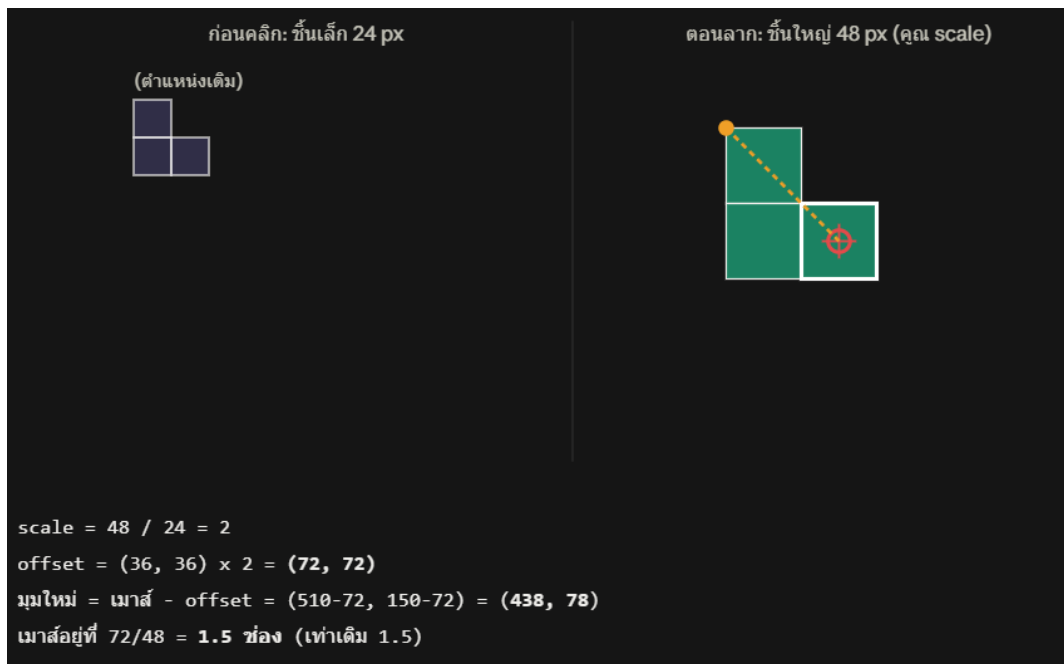
mousePressed()

ทำงานเมื่อผู้เล่นกดเมาส์ลง มีหน้าที่หาว่าคลิกโดนชิ้นส่วนไหนในมือ แล้วเตรียมชิ้นนั้นให้พร้อมลาก

1. **เช็คเกมจบ** ถ้า `game_over` เป็นจริง เรียก `setup()` เริ่มเกมใหม่ แล้ว `return` ออกทันที ไม่ต้องหาชิ้นส่วนต่อ
2. **หาชิ้นส่วนที่คลิกโดน** วนดู `hand` ที่ละช่อง ซ้ำมช่องที่เป็น 0 (ว่าง) แล้วเช็คจาก `contains_point(mouseX, mouseY)` ว่าเมาส์อยู่บนชิ้นนั้นไหม
3. **กรณีมีการคลิกโดนชิ้นส่วน** ให้เก็บชิ้นส่วนไว้ในตัวแปร `selected_piece` และเก็บลำดับช่องไว้ใน `selected_index`
4. **คำนวณ offset** หาระยะจากมุมซ้ายบนของชิ้นส่วนถึงเมาส์ เพราะ `draw()` วาดโดยเริ่มจากมุมนั้น จึงต้องรู้ว่าเมาส์อยู่ห่างจากเมาส์เท่าไร แล้วคงระยะนี้ไว้ตลอดการลาก ชิ้นส่วนจึงติดเมาส์ตอนลากได้ แล้วคูณ `scale` ($48 / 24 = 2$) เพราะตอนลากชิ้นส่วนจะขยายเป็น 2 เท่า ระยะก็ต้องยาวขึ้น 2 เท่า ด้วย เมาส์จะได้ยังอยู่บน `block` เดิม
5. **เริ่มสถานะลาก** ตั้ง `is_dragging = True` ให้ `draw()` วาดเป็นขนาดใหญ่ แล้วย้ายมุมซ้ายบนไปที่ตำแหน่ง `mouse - offset` ทันที
6. **return** เจอชิ้นที่คลิกแล้ว ไม่ต้องวนดูช่องที่เหลือ

ถ้าวนครบทุกช่องแล้วไม่โดนชิ้นไหน จะไม่เกิดอะไรขึ้น `selected_piece` ยังเป็น `None` จากนั้นหลังจากนั้น ทุกครั้งที่เมาส์ขยับ `mouseDragged()` คำนวณมุมใหม่ด้วย `mouse - offset`





mouseDragged():

ทำงานซ้ำทุกครั้งที่มีเมาส์ขยับขณะกดค้าง มีหน้าที่ย้ายชิ้นส่วนให้เคลื่อนตามเมาส์

1. เช็ควากำลังถือชิ้นส่วนอยู่ไหม if selected_piece != None ถ้าผู้เล่นกดที่ว่างแล้วลาก selected_piece จะเป็น None ไม่มีอะไรให้ย้าย
2. คำนวณมุมซ้ายบนใหม่ ใช้ตำแหน่งเมาส์ปัจจุบัน - drag_offset ที่ mousePressed() คำนวณไว้ ตอนคลิก (แกน x = mouseX - drag_offset_x, แกน y = mouseY - drag_offset_y)
3. ผลที่ได้ ระยะระหว่างมุมชิ้นส่วนกับเมาส์คงที่ตลอดการลาก

Piece.reset_pos (self):

ส่งชิ้นส่วนกลับไปอยู่ที่เดิมในมือ เมื่อผู้เล่นปล่อยชิ้นส่วนในตำแหน่งที่วางไม่ได้

1. คืนตำแหน่ง ตั้ง x, y (ตำแหน่งปัจจุบัน) ให้เท่ากับ anchor_x, anchor_y (ตำแหน่งที่จำไว้ตอนสร้างชิ้นส่วน) ชิ้นส่วนจึงกลับไปช่องเดิมในมือ
2. ยกเลิกสถานะลาก ตั้ง is_dragging = False ให้ draw() กลับไปวาดเป็นขนาดเล็ก (24 px) และ contains_point() ตรวจสอบขนาดเล็ก ผู้เล่นจึงคลิกจับใหม่ได้

Description “Commit 5”

- มีการเขียนการทำงานโค้ดเพิ่มเติม ใน function ต่อไปนี้

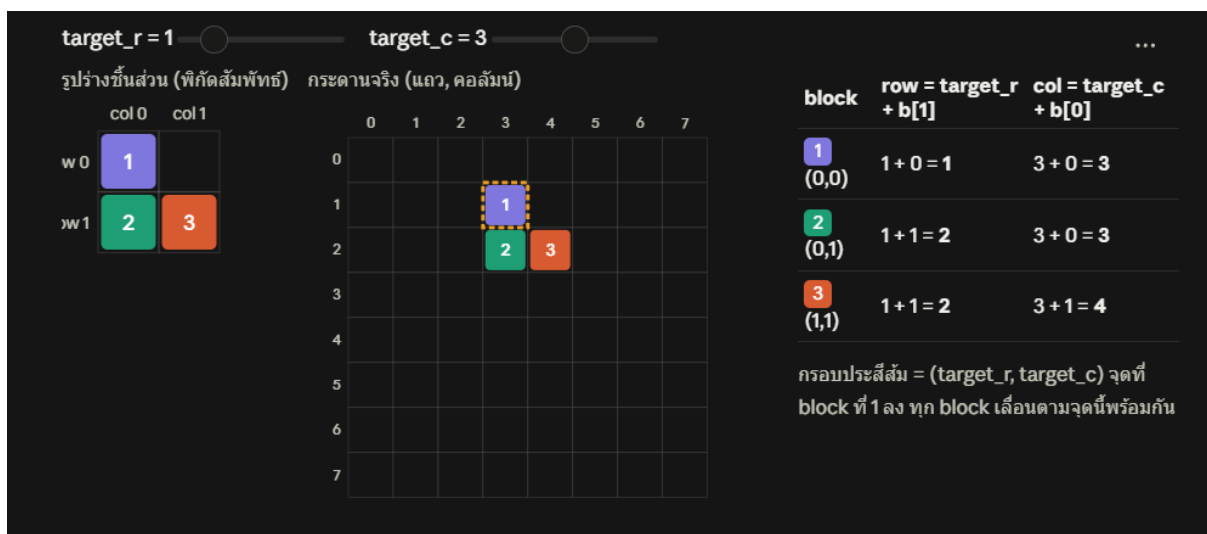
`Board.can_place()` - ตรวจสอบว่าสามารถวาง piece ลงบนกระดานโดยให้บล็อกแรกอยู่ที่ `(target_r, target_c)` ได้หรือไม่

`Board.place()` - วางชิ้นส่วน piece ลงบนกระดานจริง

`Board.can_place(self, piece, target_r, target_c):`

ตรวจสอบว่าวางชิ้นส่วนลงบนกระดานได้ไหม โดยให้ block แรก (0, 0) ลงที่ช่อง `(target_r, target_c)` คืนค่า True ถ้าวางได้ และ False ถ้าวางไม่ได้ ฟังก์ชันนี้แค่ตรวจสอบ ไม่แก้ไขข้อมูลใน grid

1. กำหนดให้สามารถวางได้ก่อน ตั้ง `ok = True` แล้วค่อยเปลี่ยนเป็น False เมื่อเจอ block ที่ผิดเงื่อนไข
2. วนตรวจทีละ block ชิ้นส่วนหนึ่งชิ้นมีหลาย block และแต่ละตัวลงคนละช่องในตาราง จึงต้องตรวจสอบครบทุกตัว เงื่อนไขของ while มี `and ok` จะหยุดทันทีที่เจอตัวที่วางไม่ได้
3. แปลงเป็นช่องจริงบนกระดาน โดยมีการคำนวณแถวและคอลัมน์ `row = target_r + blocks[i][1]` และ `col = target_c + blocks[i][0]` เอาพิกัดสัมพันธ์ของ block บวกกับจุดเริ่มวาง



4. ตรวจ 2 อย่าง

- หลุดขอบกระดานไหม (row หรือ col น้อยกว่า 0 หรือมากกว่าหรือเท่ากับ self.size) ถ้าหลุด ok = False
- ช่องนั้นมี block อื่นอยู่แล้วไหม (grid[row][col] != 0) ถ้ามี ok = False

5. หลังลูปจบ return ok

Board.place(self, piece, target_r, target_c):

วางชิ้นส่วนลงกระดานจริง โดยเขียนเลขสีลงในช่องของ self.grid ทุกช่องที่ชิ้นส่วนครอบคลุม

1. คำนวณเลขสี color_value = piece.color_idx + 1 บวก 1 เพราะต้องการกำหนดค่าสีจริง ซึ่งมีลำดับสี 1-5 ดังนี้ 1.Orange-Red 2 .Blue 3.Green 4.Yellow 5.Purple
2. วนทีละ block ชิ้นส่วนหนึ่งมีหลาย block
3. แปลงเป็นช่องบนกระดานจริง row = target_r + blocks[i][1] และ col = target_c + blocks[i][0] เอาพิกัดสัมพัทธ์บวกกับจุดเริ่มวาง
4. เขียนสีลงช่อง self.grid[row][col] = color_value ซึ่งช่องนั้นเปลี่ยนจาก 0 เป็นเลขสี แล้ว Board.draw() จะวาดเป็นช่องสีตามเลขนี้